

Recommending Refactoring Operations in Large Software Systems

Gabriele Bavota, Andrea De Lucia, Andrian Marcus, and Rocco Oliveto

Abstract During its life cycle the internal structure of a software system undergoes continuous modifications. These changes push away the source code from its original design, often reducing its quality. In such cases **refactoring** techniques can be applied to improve the readability and reducing the complexity of source code, to improve the architecture and provide for better software extensibility. Despite its advantages, performing refactoring in large and non-trivial software systems can be very challenging. Thus, a lot of effort has been devoted to the definition of automatic or semi-automatic approaches to support **developers** during software refactoring. Many of the proposed techniques are for recommending refactoring operations. In this chapter we present *guidelines* on how to build such recommendation systems and how to evaluate them. We also highlight some of the *challenges* that exist in the field, pointing towards future research directions.

1 Software Refactoring

During software evolution change is the rule rather than the exception [27]. Continuous modifications in the environment and requirements drive software evolution. Unfortunately, programmers do not always have the necessary time to make sure that the changes conform to good design practices. In consequence software quality

Gabriele Bavota
University of Sannio, Benevento (Italy), e-mail: gbavota@unisannio.it

Andrea De Lucia
University of Salerno, Fisciano (Italy), e-mail: adelucia@unisa.it

Andrian Marcus
Wayne State University, Detroit MI (USA), e-mail: amarcus@wayne.edu

Rocco Oliveto
University of Molise, Pesche (Italy), e-mail: rocco.oliveto@unimol.it

often decreases, resulting in more difficulties in maintaining existing software [45]. Several empirical studies have provided evidence that low design quality is generally associated with lower productivity, more rework, and more effort for developers [7, 17, 21]. *Refactoring* is one solution aimed at restoring or improving the quality of the software.

Refactoring has been defined as “*the process of changing a software system in such a way that it does not alter the external behavior of the code yet improves its internal structure*” [27]. Different refactoring operations might improve different quality aspects of a system and help in removing *code bad smells* [27] (i.e., symptoms of possible design problems in source code) as well as *antipatterns* [19] (i.e., design flaws in source code)¹. As an example, in Object-Oriented systems Blob classes, i.e., large classes implementing unrelated responsibilities, can be decomposed by splitting their methods into different classes that group together strongly related responsibilities and are easier to comprehend and maintain (this operation is known as *Extract Class* refactoring). Typical advantages of refactoring include improved readability and reduced complexity of source code, a more expressive internal architecture and better software extensibility [27]. Refactoring is advocated as a good programming practice to be continuously performed during software development and maintenance [15, 27, 32, 43]. In fact, as explained by Kerievsky, “*by continuously improving the design of code, we make it easier and easier to work with. This is in sharp contrast to what typically happens: little refactoring and a great deal of attention paid to expediently adding new features. If you get into the hygienic habit of refactoring continuously, you’ll find that it is easier to extend and maintain code*” [32].

Despite its advantages, performing some refactoring operations in large and non-trivial software systems can be very challenging. First, identifying refactoring opportunities in large systems is very difficult, as the design flaws are not always obvious [27]. Second, when a design problem has been identified, it is not always easy to apply the correct refactoring operation to solve it. As an example, splitting a non-cohesive class into different classes with strongly related responsibilities (i.e., *Extract Class* refactoring) requires the analysis of all the methods of the original class to identify groups of methods implementing similar responsibilities, which should be grouped together in the new classes to be extracted. This task becomes harder when the size of the class to split increases and its cohesion decreases. In the end, once the refactoring solution has been defined, the software engineer must apply it without changing the external behavior of the system. All these observations highlight the need for refactoring recommendation systems supporting the software engineer in (i) identifying refactoring opportunities (i.e., design flaws) and (ii) designing and applying a refactoring solution.

Regarding the identification of refactoring opportunities, several approaches specialized in the identification of design flaws in source code (e.g., antipattern) have

¹ The main difference between code smells and anti-patterns is that a code smell represents something “probably wrong” in the code, while an anti-pattern is certainly a design problem in source code. In other words, a code smell might indicate an antipattern. As example, a Large Class (bad smell) is one of the symptoms of a Blob Class (antipattern).

been proposed in the literature. Some of these approaches are manual [65], others apply rules based on software product metrics [33, 42, 48] or other source of information (e.g., design change-propagation [58]). These approaches are out of the scope of this chapter, since they are focused on identifying a code design problem, ignoring how to solve it through an appropriate refactoring operation. The interested reader can find a complete treatment of more recent work in this field in the paper by Moha et al. [48].

On the other hand, approaches recommending how to refactor a design flaw are the focus of this chapter². By analyzing these approaches we will derive a set of guidelines for building and evaluating recommendation systems for supporting software refactoring. In addition, we will also discuss some problems that are still open, which will allow us to start tracing future research directions in the field.

2 Recommendation Systems for Software Refactoring

There are more than 90 different refactoring operations defined in the Fowler's catalog³. Most of the refactoring operations are simple source code transformations aimed at increasing source code comprehension. For example, the *Remove Parameter* refactoring removes a parameter not used anymore by the method. Most research efforts have been focused on providing support to developers when performing more complex refactoring operations, e.g., *Extract Class* [27]. In the following we discuss the recommendation systems existing in literature classifying them on the basis of the supported refactoring operation(s). Specifically, for each approach we report: (i) the support it provides to perform the refactoring operation(s), (ii) the information it exploits from source code, (iii) the algorithm used, and (iv) the evaluation conducted for it. Table 1 summarizes the refactoring operations discussed in this chapter, by classifying them based of the benefits provided to the source code.

2.1 Improving code decomposition

In this section we present recommendation systems supporting refactoring operations aimed at improving code decomposition.

2.1.1 Extract Class Refactoring

Extract Class refactoring is used to remove the Blob antipattern [19] from a software system. A Blob is a large and complex class that centralizes the behavior of a portion

² Note that some of these approaches are also able to identify the design flaw besides suggesting how to solve it.

³ <http://refactoring.com/catalog/>

Table 1: Refactoring operations discussed in this Chapter

Name	Description	References
<i>Improving code decomposition</i>		
Extract Class	Splits a complex and low cohesive class into new classes having a better (focused) defined set of responsibilities.	[10, 13, 25, 63]
Extract Package	Splits a package grouping heterogeneous responsibilities into different cohesive packages grouping together classes implementing similar responsibilities.	[9]
Extract Method	Splits a method composed by several code fragments performing different actions, into new methods, each one implementing a specific and isolable action.	[1, 67]
<i>Improving names and location of code</i>		
Rename Method (Field)	Applied when the name of a method (field) does not reveal its purpose, compromising the code comprehension.	[4]
Move Method	Moves a method m from its original class to an “envied class”, containing responsibilities closer to those implemented by m .	[5, 52, 62, 66]
Move Class	Moves a class C from its package to a new package grouping together responsibilities closer to those implemented by C .	[3, 12]
<i>Improving conformance with Object-Oriented programming principles</i>		
Push Down Field	In a class hierarchy, moves a field only used by some subclasses from the superclass to the subclasses using it.	[51]
Pull Up Field	In a class hierarchy, moves a field implemented in all subclasses to their superclass.	[51]
Push Down Method	In a class hierarchy, moves a method only used by some subclasses from the superclass to the subclasses using it.	[51]
Pull Up Method	In a class hierarchy, moves a method implemented in all subclasses to their superclass.	[51]
Extract Hierarchy	When a class is implementing several different behaviors in conditional statements, extract from it a hierarchy of classes, each one representing a special case.	[51]
Collapse Hierarchy	Merges together a superclass and the subclass that are very similar.	[51]

of a system and only uses other classes as data holders, i.e., data classes. Blobs are generally characterized by low cohesion since they implement several different responsibilities. In addition, due to the numerous dependencies with the data classes, Blobs also exhibit high levels of coupling.

Extract Class refactoring is applied to split the responsibilities implemented in a Blob class into different classes with higher cohesion (i.e., grouping together strongly related responsibilities). Blob classes have negative impact on comprehension and maintenance activities, well documented by existing empirical studies [7, 17, 21, 29, 41]. Hence removing Blobs is a very important design improve-

ment activity. Given the complex nature of Blob classes, performing Extract Class refactoring is a difficult task, as the following steps must be performed:

1. Analyzing the methods of the Blob class (which are often in the hundreds) to understand the main responsibility for each of them;
2. Identifying clusters of methods that implement similar and related responsibilities;
3. Distributing the attributes of the Blob class among the identified clusters of methods;
4. Splitting the Blob class into new classes, which contain the clusters of methods and attributes;
5. Ensuring that no changes in the system behavior result from this refactoring.

Manual Extract Class refactoring is a difficult and error prone task, hence research has been devoted to define approaches able to support the developers in performing extract class refactoring.

The Extract class refactoring can be formulated as a **cluster analysis** problem, where it is necessary to identify the optimal partitioning on methods in different classes. As proposed by Fokaefs et al. [26], structural dependencies between the entities of a class to be refactored, i.e., attributes and methods, can be exploited to guide the clustering process. Specifically, using this information, Fokaefs et al. [26] compute its *entity set* for each attribute, i.e., the set of methods using it. For each method they compute also the *entity set*, i.e., all the methods that are invoked by a method and all the attributes that are accessed by it. The Jaccard distance between all couples of entity sets of the class is computed in order to cluster together cohesive groups of entities that can be extracted as separate classes. The Jaccard distance is computed as follows:

$$Jaccard(E_i, E_j) = 1 - \frac{|E_i \cap E_j|}{|E_i \cup E_j|}$$

where E_i and E_j are two entity sets, the numerator is the number of common entities between the two sets and the denominator is the total number of unique entities in the two sets.

The approach proposed by Fokaefs et al. [26] has been implemented as an Eclipse plug-in, called JDeodorant⁴. The benefits of JDeodorant has been empirically analyzed. The empirical evaluation indicated that the refactoring operations provided by JDeodorant are meaningful and they approximate the refactoring operations previously performed by three developers with 67% precision and 82% recall.

An alternative approach for the identification of Extract class refactoring operations is based on graph theory, and in particular on the concept of a transitive closure [13]. The approach takes as input a class previously identified as a candidate for refactoring. The class is refactored following the process depicted in Fig. 1. In the top path of the process the candidate class is parsed to build a *method-by-method* matrix, a $n \times n$ matrix where n is the number of methods in the class to

⁴ <http://jdeodorant.com>

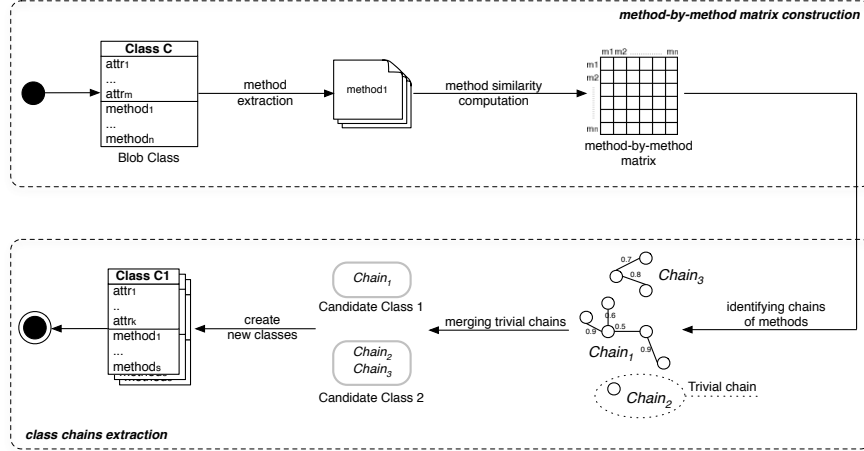


Fig. 1: Class extraction process based on the transitive closure of graphs proposed by Bavota et al. [13]

be refactored. A generic entry $c_{i,j}$ of the method-by-method matrix represents the likelihood that method m_i and method m_j should be in the same class. This likelihood is computed as a hybrid coupling measure between methods (which reflects the degree to which they are related) obtained by combining three structural and semantic measures, i.e., the Structural Similarity between Methods (SSM) [28], the Call-based Dependence between Methods (CDM) [10], and the Conceptual Similarity between Methods (CSM) [55]. In the SSM measure the higher the number of instance variables shared by two methods, the higher their likelihood to be in the same class. CDM takes into account the calls performed by the methods, i.e., the higher the calls interaction between two methods the higher their coupling. Finally, CSM measures the coupling between methods as their textual similarity. The conjecture is that if developers used similar terms in comments and identifiers of two methods, it is likely that they are “describing” similar responsibilities implemented by the two methods. These three measures are combined through a weighted sum in order to obtain the hybrid coupling measure mentioned before. It is worth noting that the *method-by-method* matrix is just a convenient way of storing a weighted graph representing the Blob class, where each node in the graph represents one of its methods and the weight of the edge connecting two nodes (i.e., methods) represents their coupling.

Using the information in the *method-by-method* matrix, the second part (bottom path) of the refactoring process shown in Fig. 1 extracts the new classes from the input Blob. In particular, a filtering step is used to remove spurious links and to split the initial graph represented in the *method-by-method* matrix into disconnected sub-graphs. This is done by removing all edges in the graph having a weight lower than a defined threshold named *minCoupling*. Then, the approach identifies the chains

```

public class UserManagement {
    private static final String TABLE_USER = "user";
    private static final String TABLE_TEACHING = "teaching";
    private static final String TABLE_ROLE = "role";

    public void insertUser(User pUser){
        boolean check = checkMandatoryFieldsUser(pUser);
        String sql = "INSERT INTO " + UserManagement.TABLE_USER + " ... ";
        ...
    }

    public void updateUser(User pUser){
        boolean check = checkMandatoryFieldsUser(pUser);
        String sql = "UPDATE " + UserManagement.TABLE_USER + " ... ";
        ...
    }

    public void deleteUser(User pUser){
        String sql = "DELETE FROM " + UserManagement.TABLE_USER + " ... ";
        ...
    }

    public void existsUser(User pUser){
        String sql = "SELECT FROM " + UserManagement.TABLE_USER + " ... ";
        ...
    }

    public boolean checkMandatoryFieldsUser(User pUser){
        ...
    }

    public void insertTeaching(Teaching pTeaching){
        boolean check = checkMandatoryFieldsTeaching(pTeaching);
        String sql = "INSERT INTO " + UserManagement.TABLE_TEACHING + " ... ";
        ...
    }

    public void updateTeaching(Teaching pTeaching){
        boolean check = checkMandatoryFieldsTeaching(pTeaching);
        String sql = "UPDATE " + UserManagement.TABLE_TEACHING + " ... ";
        ...
    }

    public void deleteTeaching(Teaching pTeaching){
        String sql = "DELETE FROM " + UserManagement.TABLE_TEACHING + " ... ";
        ...
    }

    public boolean checkMandatoryFieldsTeaching(Teaching pTeaching){
        ...
    }

    public void insertRole(Role pRole){
        boolean check = checkMandatoryFieldsRole(pRole);
        String sql = "INSERT INTO " + UserManagement.TABLE_ROLE + " ... ";
        ...
    }

    public void deleteRole(Role pRole){
        String sql = "DELETE FROM " + UserManagement.TABLE_ROLE + " ... ";
        ...
    }

    public boolean checkMandatoryFieldsRole(Role pRole){
        ...
    }
}
    
```

Fig. 2: UserManagement: an example of Blob class [13]

of connected methods belonging to the different subgraphs. Each computed chain represents a class to be extracted from the original class. However, some of these chains could have a very short length (trivial chains). To avoid the extraction of classes with a very low number of methods, each trivial chain is merged with the most coupled non trivial chain to obtain the final set of classes to be extracted from the original class. The attributes of the original class are also distributed among the extracted classes according to how they are used by the methods in the new classes, i.e., each attribute is assigned to the new class having the higher number of methods using it.

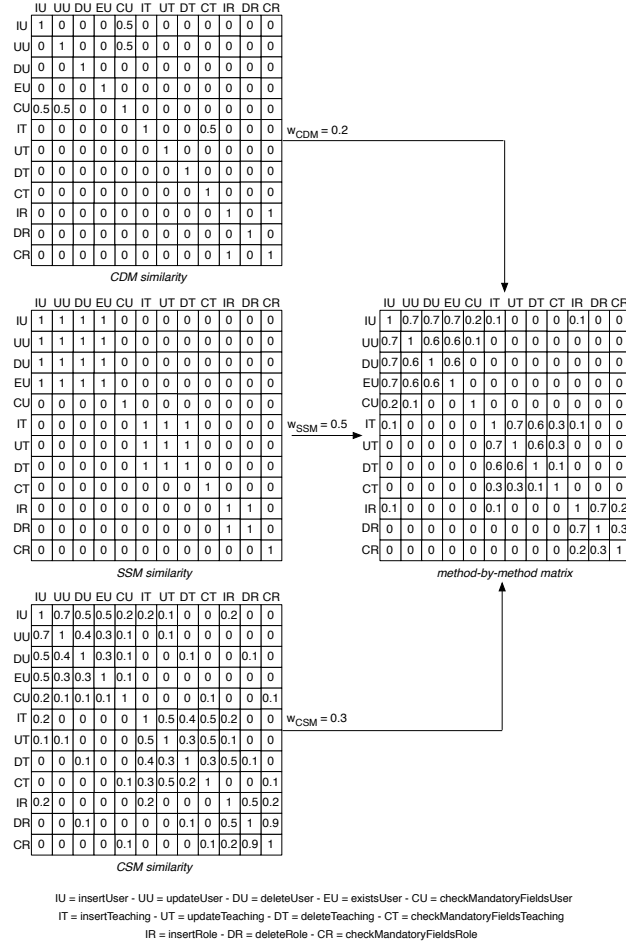


Fig. 3: Method-by-method matrix construction [13]

The approach has been implemented as an Eclipse plug-in, called ARIES [14]. To better understand how ARIES [13, 14] works, let us assume that we are interested in refactoring the *UserManagement* class shown in Fig. 2. Given its name and its set of methods, probably the original responsibility of this class was to implement a set of operations that allow to manipulate the *User* entity in the database. However, during software maintenance, two new responsibilities were added to this class, i.e., the management of the *Teaching* entity and the management of the *Role* entity. Fig. 3 shows: (i) the values for the similarity measures used by the approach, i.e., CDM, CSM, and SSM, in three separate matrices, and (ii) how these values are combined in the *method-by-method* matrix through a weighted sum.

Fig. 4 shows how ARIES [13, 14] extracts from the *UserManagement* class three new classes having better defined responsibilities than the original class. The first

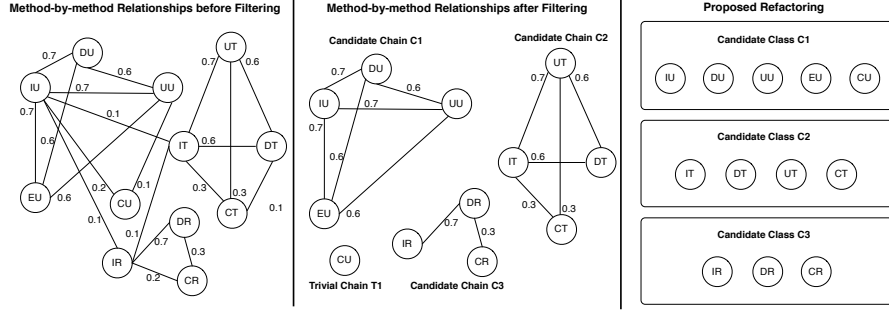


Fig. 4: The extraction process followed by ARIES [13, 14]

part of the figure shows the graph that can be obtained from the method-by-method matrix (note that the edges weighted with 0.0, i.e., pairs of methods having zero coupling, are omitted), while the second part of the figure shows the connected components obtained after the matrix filtering. In this example we arbitrary set $minCoupling = 0.2$. Thus, all the edges having weight lower than 0.2 (that represent spurious relationships between methods) are removed from the graph. The extracted components correspond to the preliminary method chains. The third part of the figure shows the refinement of the method chains. In particular, a trivial chain composed of only one method (*checkUser*) is added to the most coupled non trivial chain (i.e., C_1). In the end, ARIES recommends splitting the original class into three new classes.

ARIES aims at guiding the developer through the entire refactoring process. In fact, ARIES provides support for: (i) identifying Blob classes candidate for Extract Class refactoring, (ii) recommending the refactoring solution (i.e., how to split the Blob class), and (iii) applying the refactoring, while preserving the system behavior. The recommended refactoring solutions have been empirically evaluated through quality metrics and user studies [13] and the results show that ARIES’s recommendations: (i) strongly increase the cohesion of the refactored classes without leading to significant increases in terms of coupling; (ii) are considered useful by developers performing extract class refactoring; and (iii) are able to approximate manually performed refactoring operations with 91% accuracy.

2.1.2 Extract Package Refactoring

Extract Package refactoring is similar to Extract Class refactoring, but it acts at a different level of granularity. As it is possible to have complex classes grouping together unrelated responsibilities (i.e., Blobs), it is also possible to observe the same phenomenon at a higher granularity level: packages grouping together classes implementing unrelated responsibilities. These packages are known as “Promiscuous

packages”⁵ and usually have low cohesion. The aim of Extract Package refactoring is to split a promiscuous package into different cohesive packages grouping together classes implementing similar responsibilities. Promiscuous packages often implement entire subsystems grouping together hundreds of classes. As with the Extract Class refactoring, in order to perform Extract Package refactoring, the developer needs to analyze all classes in the promiscuous package, understand their responsibilities, and group together in new packages those implementing similar things. This is a difficult and time-consuming task. To provide support for this refactoring, some of the techniques proposed for software re-modularization could be easily adapted (see e.g., [47, 56]). One way is to consider the classes of the promiscuous package as the entire set of classes to re-modularize and apply one of the approaches existing in literature to organize the classes into new packages (i.e., to split the promiscuous package into new ones).

Alternatively, the approach proposed by Bavota et al. [9] for identifying Extract class refactoring operations can be customized for recommending Extract Package refactoring solutions as well. Using a graph theory-based algorithm (similar to the one used for Extract Class refactoring in ARIES) it is possible to exploit structural and semantic relationship between classes to extract chains of strongly related classes in the package to be refactored. The classes of the original package are distributed in different packages according to the extracted chains. If the number of extracted chains is one, then no re-modularization is recommended by the tool (this generally happens when the cohesion of the analyzed package is high). Otherwise, based on the extracted class chains, the approach recommends new packages with higher cohesion than the original package. Note that the aim of this approach is to suggest how to split a package (previously identified by the software engineer as a candidate for re-modularization) in smaller, more cohesive packages, while the implementation of the refactoring in the software system is left to the developer. However, it is worth noting that preserving the system behavior for refactoring operations acting at class level (like extract package refactoring) is quite simple, since it is enough to update package declarations and imports in the impacted classes (i.e., the moved ones and those using them). The evaluation of the approach has been performed on five systems. On four of them, original developers have been involved in the evaluation of the meaningfulness of the recommended refactoring operations, while on the fifth one external developers performed the same task. Overall, developers evaluated as meaningful 77% of the recommended refactoring operations [9].

2.1.3 Extract Method Refactoring

Extract Method refactoring is generally applied on methods composed by several code fragments performing different actions. In particular, code fragments implementing a specific and isolable action are extracted and grouped into a new method. Smaller methods increase the chance of code-reuse and ease code comprehension.

⁵ <http://www.refactoring.com/catalog/extractPackage.html>

Identifying Extract Method refactoring opportunities is far from trivial. The analysis of the history of a software system can be worthwhile for supporting such tasks. The conjecture is that statements in a method that change together are good candidate to be extracted from the original method aiming at creating a new independent method [44]. In order to identify the set of methods that change together, it is possible to use a weighted dependence graph (WPDG) [44], which is an extended program dependence graph (PDG) where edges are weighted based on the modification histories of the methods. Specifically, each node of the graph represents a statement of the method under analysis, while the weight of the edge that connect two statements i and j represents the likelihood that when changing i the developer also changes j . Once the WPDG is obtained, a threshold is used to distinguish which edges ought to remain or vanish. It is worth noting that eliminating edges with a weight lower than the fixed threshold results in splitting the original WPDG in several several subgraphs. Then, the new methods are obtained considering every reachable path from the method entry node on the split WPDG [44].

It is clear that when historical information are not available or they are poor, it is not possible to accurately identify Extract Method refactoring operations. An alternative source of information for identify such refactoring operations can be derived by using slicing [68]. A slice is an executable subset of program statements that preserves the original behavior of the program with respect to a subset of variables of interest and at a given program point [68]. It can be computed both statically or dynamically. In the context of refactoring, fine slicing is particularly suitable for identifying Extract Method refactoring operations [1]. Fine slicing is a technique to compute executable program slices and can be used to extract non-contiguous pieces of code and untangle loops [2]. The extracted slices can be used to decompose a long method body into different methods, each with a precise responsibility.

Other than fine slicing, more sophisticated slicing-based techniques can be used to identify Extract Method refactoring operations. As proposed by Tsantalis and Chatzigeorgiou [67], Extract Method refactoring solutions can be identified by employing two slicing techniques:

1. A *complete computation slice* is used to identify all the statements in a method affecting the computation of a given variable.
2. An *object state slice* is used to capture the statements affecting the state of a given object.

The proposed approach has been implemented as a fully automated technique in the JDeodorant Eclipse plugin. The evaluation, performed with a designer, shows that the proposed technique is able to capture slices of code implementing a distinct and independent functionality compared to the rest of the original method and thus leads to extracted methods with useful functionality.

2.2 Improving names and code location

Other refactoring operations aimed at improving names and code locations. Concerning the former, having meaningful names for methods and fields is essential for easier code comprehension [36]. Thus, *Rename Method (Rename Field)* refactoring should be applied when the name of a method (field) does not reveal its purpose, compromising the code comprehension. Abebe and Tonella [4] propose an approach that could be used to support these refactorings. Their technique exploits ontological concepts and relations automatically extracted from the source code to suggest identifiers names. While the approach has not been evaluated in the context of refactoring, it represents a good starting point for *Rename Method (Rename Field)* recommendation systems.

Refactorings improving the location of code aim at grouping together similar responsibilities in order to ease code comprehension and maintenance. Examples of such refactoring operations are the Move Method refactoring, Move Field refactoring, and Move Class refactoring. Move Field is applied when a field is used by another class more than the class on which it is defined [27]. However, in this family of refactorings, most researchers focused on Move Method and Move Class refactoring.

2.2.1 Move Method Refactoring

This refactoring is targeted to solve the bad smell known in literature as “Feature Envy” [27]. This smell arises when a method seems to be more interested in a class other (known as envied class) than the one it is implemented in, e.g., the method invokes getter methods of another object many times [27]. Usually, this negatively influences the cohesion and the coupling of the class in which the method is implemented. In fact, the method suffering of feature envy reduces the cohesion of the class because it likely implements different responsibilities with respect to those implemented by the other methods of the class and increases the coupling, due to the many dependencies with methods of the envied class.

By applying the Move Method refactoring the method is moved to the envied class. Unfortunately, not all the cases are cut-and-dried. Often a method uses features of several classes, thus the identification of the appropriate envied class (as well as the method to be moved) is not always trivial especially in large software systems [66]. For this reason, approaches to support move method refactoring have been proposed by researchers.

A simple way to identify Move Method refactoring operations can be obtained by analyzing the abstract syntax tree (AST) of a software system [5]. Specifically, for each field in a given class it is possible to determine the set of methods referencing the field by traversing the AST. If a foreign method makes too many references to a number of distinct fields, then the method should be moved into the class under analysis. An empirical evaluation of this approach—conducted on ten C++ systems—indicates that 88% of the recommended refactorings were meaningful.

```

extractMoveMethodRefactoringSuggestions(Method  $m$ )
     $T = \{\}$ 
     $S = \text{entity set of } m$ 
    for  $i = 1$  to size of  $S$ 
         $\text{entity} = S[i]$ 
         $T = T \cup \{\text{entity.ownerClass}\}$ 
    sort( $T$ )
     $\text{suggestions} = \{\}$ 
    for  $i = 1$  to size of  $T$ 
        if ( $T[i] \neq m.\text{ownerClass} \wedge \text{modifiesDataStructureInTargetClass}(m, T[i]) \wedge$ 
             $\text{preconditionsSatisfied}(m, T[i])$ )
             $\text{suggestions} = \text{suggestions} \cup \{\text{moveMethodSuggestion}(m \rightarrow T[i])\}$ 
    if  $\text{suggestions} \neq \emptyset$ 
        return  $\text{suggestions}$ 
    else
        for  $i = 1$  to size of  $T$ 
            if  $T[i] = m.\text{ownerClass}$ 
                return  $\{\}$ 
            else if  $\text{preconditionsSatisfied}(m, T[i])$ 
                return  $\{\text{moveMethodSuggestion}(m \rightarrow T[i])\}$ 
    return  $\{\}$ 
    
```

Fig. 5: Algorithm used for the extraction of Move Method refactoring operations proposed by Tsantalis *et al.* [66]

Alternatively, the identification of Move Method refactoring could be formulated as an optimization problem, i.e., finding a location for a specific method that optimize some quality metrics, such as cohesion and coupling [62]. Since the search space is huge, meta-heuristic search algorithms, such as Genetic Algorithms (GA), can be used to find a pseudo-optimal solution. An empirical evaluation conducted on an open source system confirms the suitability of GA for identifying Move Method refactoring operations [62].

Note that the two approaches described above are just meant to recommend the refactoring operation to perform and not to apply it on the software system (by ensuring behavior preservation). A fully automated approach for move method refactoring has been proposed by Tsantalis and Chatzigeorgiou [66]. The approach, implemented in the JDeodorant Eclipse plugin, is able to identify move method refactoring operations as well as to apply them in the system by preserving the behavior. The algorithm used by their approach is shown in Fig. 5. Given a method m , the approach forms a set of candidate target classes where m should be moved (set T in Fig. 5). This set is obtained by examining the entities (i.e., attributes and methods) that m accesses from the other classes (entity set S in Fig. 5). In particular, each class in the system containing at least one of the entities accessed by m is added to T . Then, the candidate target classes in T are sorted in descending order according to the number of entities that m accesses from each of them ($\text{sort}(T)$ in Fig. 5). In the following steps each target class $T[i]$ is analyzed to verify its suitability to be the recommended class. In particular, $T[i]$ must satisfy three conditions to be considered in the set of candidate suggestions: (i) $T[i]$ is not the class m currently belongs to,

(ii) m modifies at least one data structure in $T[i]$, and (iii) moving m in $T[i]$ satisfies a set of behavior preserving preconditions (see [66] for the complete description of these preconditions). The set of classes in T satisfying all the above described conditions are put in the *suggestions* set (see Fig. 5). If *suggestions* is not empty, the approach suggests to move m in the first candidate target class following the order of the sorted set T . On the other side, if *suggestions* is empty, the classes in the sorted set T are again analyzed by applying milder constraints than before. In particular, if a class $T[i]$ is the m owner class, then no refactoring suggestion is performed and the algorithm stops. Otherwise, the approach checks if moving the method m into $T[i]$ satisfies the behavior preserving preconditions. If so, the approach suggests to move m into $T[i]$. An empirical evaluation conducted on open source systems indicate that the refactoring operations proposed by the approach make sense from the point of view of the designer involved in the evaluation.

The approach described above use only structural information (e.g., method calls) for identifying Move Method refactoring opportunities. However, analyzing identifiers and comments in the source code can derive “semantic” information that could be combined with structural information aiming at identifying more meaningful Move Method refactoring operations. This is the idea behind Methodbook [52], an approach that analyzes methods to extract words contained in comments and identifiers. Then, a light-weight static analysis is performed to detect: (i) structural dependencies between methods (i.e., method calls and shared instance variables), and (ii) the original system design. The extracted structural and semantic information is provided as input to RTM (Relational Topic Model) [20], a probabilistic topic modeling technique representing documents (i.e., methods) as random mixtures over latent topics, where each topic is characterized by a probabilistic distribution over words and is represented by a set of words mostly relevant for explaining the topic. The provided structural information are used by RTM to adjust the probability distribution of each topic taking into account explicit relationships between documents. In Methodbook, explicit relationships between documents (methods) are modeled through (i) the structural dependencies existing among the methods, and (ii) the original design. The same set of behavior preserving preconditions defined by Tsantalis *et al.* [66] are applied in Methodbook, allowing the automatic application of the suggested move method refactorings on the software system. A preliminary evaluation indicates that Methodbook was able to correctly place about 70% of the analyzed methods.

2.2.2 Move Class Refactoring

Move Class refactoring aims at solving one of the main reasons for architectural erosion in software systems: inconsistent placement of source code classes in software packages [3]. Such a scenario negatively impacts the package cohesion and also increases the number of dependencies (coupling) between packages [35]. In such cases, re-modularization of the system is necessary [27, 50]. While most of the approaches existing in literature focus on proposing a new, system level re-

modularizations to the developer (e.g., [30, 40, 56, 70]), using Move Class refactoring it is possible to perform a more focused and fine-grained re-modularization, by moving the misplaced class into a more suitable package of the system, i.e., one that has classes more functionally related to it than the one it is placed in. Identifying Move Class refactoring opportunities in a large software system is not easy, due to the number of classes contained in it and to the intricate web of relationships existing among them.

Like the Move Method refactoring, the identification of Move Class refactoring operations can be formulated as an optimization problem [3]. Specifically, starting from an initial decomposition, a search-based approach can be adopted for automatically reducing (structural) dependencies between packages of a software system by moving classes between the original packages. An empirical evaluation confirms the benefits of such an approach; by applying the recommended refactoring operations it is possible to achieve improvements in package cohesion together with coupling reduction.

It is worth noting, that besides structural information, semantic information can also be used to identify Move Class refactoring operations [12]. Specifically, the analysis of underlying latent topics in source code (acquired via RTM) can be combined with structural dependencies aiming at identifying more meaningful refactoring operations. An empirical evaluation confirms the benefits of such a combination. Specifically, the combined approach provides not only a coupling reduction among the software modules, but also recommendations that are considered by developers meaningful from a functional point of view [12].

Note that, as explained for the Extract Package refactoring, all approaches recommending move class refactoring operations can be easily automated to preserve the system behavior due to the fact that this refactoring acts at class level.

2.3 Improving Conformance with Object-Oriented Programming Principles

All refactoring operations aimed at improving encapsulation, inheritance, and polymorphism fall in this category.

For example, Replace Conditional With Polymorphism refactoring is applied when in a class there is a conditional expression choosing a different behavior depending on the type of an object [27]. Applying this refactoring (i) the method containing the conditional expression is converted in an abstract method, and (ii) each branch of the conditional expression is moved into an overriding method in a subclass. In this way, it is possible to benefit of all advantages derived by polymorphism.

Encapsulate Field refactoring is used to make private a field, providing getters and setters method to access it from the outside (if needed). This refactoring improves the encapsulation of the code by ensuring that all accesses to a field are managed by explicitly designed methods.

Other refactoring operations improve the inheritance organization of objects. As example, Pull Up Method (Pull Up Field) refactoring is used when two subclasses inheriting from the same class implement exactly the same method (field) while Push Down Method (Push Down Field) refactoring is used when a method (field) implemented in a superclass is of interest only for some of its subclasses [27].

While no techniques have been proposed in literature for supporting Replace Conditional With Polymorphism and Encapsulate Field refactoring, an approach dealing with the improvement of the inheritance organization of objects have been proposed by O’Keeffe and O’Cinneide [51]. They formulate the task of refactoring as a search problem in the space of alternative designs. The alternative designs are generated by applying a set of refactoring operations. In particular, the refactoring types considered in this work are: push down field, pull up field, pull down method, pull up method, extract hierarchy, and collapse hierarchy. The search for the optimal design is guided by a quality evaluation function based on eleven object-oriented design metrics, i.e., the Chidamber and Kemerer (CK) metrics [21] that reflects refactoring goals. The results achieved in the reported experimentation show that the presented approach is able to improve the design quality of a given system from a quality metric point of view.

3 Building a Refactoring Recommendation System

When building a refactoring recommendation system there are two critical issues to deal with:

1. *Capturing relationships existing between code components.* This step is very important since the goal of several refactoring operations (e.g., Extract Class, Move Method, Move Field, Extract Subclass, Extract Package, Pull Up Method, etc.) is to re-organize code components in such a way that related components (i.e., those implementing similar responsibilities) are grouped together. There are different sources of information that can be exploited to capture relationships between the code components object of the refactoring.
2. *Define the algorithm to generate the refactoring recommendation.* Each algorithm has strengths and weaknesses. On the basis of the refactoring operations to support, an algorithm ensuring a fair compromise of strengths and weaknesses must be chosen.

In the following section we present some guidelines on how to deal with these two issues.

3.1 Capturing Relationships between Source Code Components

The analysis of the relationships existing between code components can be made from different perspectives. Most of the techniques existing in literature exploit structural information extracted by statically analyzing the source code to capture different relations between components (e.g., classes), such as, the number of calls between two entities, variable accesses, or inheritance relations. A second option is dynamic information, which takes into account call relationships occurring during program execution. In addition, textual (i.e., semantic) information can be exploited to capture relations between code components from the code lexicon using Information Retrieval (IR) techniques [6]. Finally, historical data can be used to identify co-changing code components.

The refactoring approaches existing in literature mostly exploit structural and semantic information and not so much the dynamic and historical information⁶. This is due to the fact that structural and semantic information are easily obtainable from the source code while dynamic and historical ones are very difficult to collect and not always available. Moreover, recent work showed that using only structural and semantic information it is possible to capture almost all relationships captured with dynamic and historical information [11]. Thus, in the following, we only discuss structural and semantic relationships extracted from the source code to support software refactoring.

3.1.1 Structural Relationships

In this section we discuss the sources of information capturing structural relationships (i.e., structural coupling) between code components. Table 2 summarizes the sources of information exploited by the approaches in literature.

Method Calls. The most obvious source of information that can be exploited to capture structural relationships between code components is the calls interaction. Method Calls as source of information to capture relationships between code components have been exploited in many refactoring recommendation approaches [3, 5, 10, 12, 13, 25, 51, 52, 62, 63, 66] (see Table 2 for details). Methods generally call each other when co-operating in the implementation of some responsibilities. This source of information can be useful for refactoring operations acting at both method (e.g., Extract Class, Move Method) and class (e.g., Extract Package, Move Class) level.

Methods with high call interactions are good candidates to be grouped together when performing refactoring operations acting at method level. As example, this source of information is important for Extract Class refactoring (i.e., which methods of the class to be split should be grouped together), for Inline Method (i.e., which

⁶ The only exception is represented by the approach proposed by Maruyama and Shima [44] that exploits historical information.

Table 2: Sources of information exploited in the literature to capture relationships between code components

Reference	Supported Refactoring	Method Calls	Shared Variables	Inheritance Relationships	Original Decomposition	Semantic Similarity
[63]	Extract Class	X	X			
[25]	Extract Class	X	X			
[10]	Extract Class	X	X			X
[13]	Extract Class	X	X			X
[5]	Move Method	X				
[62]	Move Method	X	X			
[66]	Move Method	X	X			
[52]	Move Method	X	X		X	X
[1]	Extract Method		X			
[67]	Extract Method		X			
[9]	Extract Package	X				X
[3]	Move Class	X			X	
[12]	Move Class	X			X	X
[51]	Multiple Refactorings	X	X	X	X	

methods co-operate so much that merging one into another would be better to reduce coupling) and so on. A measure capturing method call interactions is the Call-based Dependence between Methods (CDM) [10]. CDM has values in $[0, 1]$; the higher the number of calls between two methods, the higher the CDM value and thus, the coupling between methods.

The calls between methods belonging to different classes also represent frequently used information to measure coupling between classes. In fact, it is reasonable to think that classes having many call interactions co-operate to implement the same (or strongly related) responsibilities and thus, are highly coupled. This information is particularly important for refactoring operations aimed at improving the modularization quality of Object-Oriented systems, e.g., Extract Package, Move Class. There are many metrics available in literature to measure the coupling between classes based on their call interactions. Examples are the Information-Flow-based Coupling (ICP) [37] and the Message Passing Coupling (MPC) [38].

Shared Instance Variables. The instance variables shared by two methods are an important source of information for refactoring operations acting at method level (e.g., Extract Class, Move Method). In fact, they also represent a form of communications between methods (performed through shared data). Thus, methods sharing instance variables are more coupled than methods not sharing any data. Shared Instance Variables as source of information to capture relationships between code components have been used in many refactoring recommendation ap-

proaches [1, 3, 10, 12, 13, 25, 51, 52, 62, 63, 66, 67] (see Table 2 for details).

A measure to capture this form of coupling between methods is the Structural Similarity between Methods (SSM) [28], used to compute the cohesion metric ClassCoh [28]. SSM has values in $[0, 1]$; the higher the number of instance variables the two methods share, the higher the SSM value is and thus, the coupling between methods.

Inheritance Relationships. A source of structural information to capture relationships between classes (and thus, useful to refactoring acting at class level) are the inheritance dependencies existing among them. Exploiting this information is mandatory when working on approaches aimed at supporting refactoring operations that modify the class hierarchy, e.g., Extract Subclass, Extract Superclass, Pull Up Method, Push Down Method. Inheritance relationships have been exploited to support refactoring operations dealing with improving class hierarchy [51] (see Table 2).

Generally, the measurement of inheritance relationships between two classes is performed through a simple boolean value: *true* if two classes have inheritance relationships or *false* otherwise.

Original Decomposition A last source of structural information that can be exploited to capture relationships between code components is the original decomposition or, in other words, the choices made by the developers when designing the system. For example, if the developers placed two methods inside the same class it is reasonable to assume that from their point of view these two methods are in some way related. For instance, in case of Move Method refactoring, this information can be used to take into account the choices made by the original developers when suggesting refactoring operations. The same conjecture can be also made at class level: if two classes were implemented in the same package by the developers it is likely that from their point of view these two classes were in some way related. Information about the original decomposition has been used for Move Method [52] and Move Class [12] refactorings (see Table 2).

3.1.2 Semantic Relationships

The semantic relationships between code components are computed by measuring their textual similarity. If the vocabulary of two code components (i.e., methods or classes) is very similar, then it is likely that the developers used similar terms to describe similar responsibilities implemented by the two components. This information can be useful to support all kinds of refactoring aimed at grouping together similar code components, at both method and class level.

IR techniques have been employed to measure the textual similarity between code components. Semantic coupling can be captured at both method and class level. The Conceptual Similarity between Methods (CSM) has been introduced by Marcus et al. [41] to define the Conceptual Cohesion of Classes (C3) [41] and the Concep-

tual Coupling Between Classes (CCBC) [55]. Two methods are conceptually related if their (domain) semantics are similar, i.e., they perform conceptually similar actions. In order to compute CSM between two methods, Marcus and Poshyvanyk propose the use of Latent Semantic Indexing (LSI) [23], an advanced IR method that can be used to compute the textual similarity between the two methods. Clearly, the higher the similarity, the higher the conceptual similarity.

Semantic relationships have been exploited in several refactoring recommendation approaches [9, 10, 12, 13, 52] (see Table 2 for details).

3.2 Define the Algorithm to Generate the Refactoring Recommendations

The choice of algorithm to be applied in order to identify a refactoring solution mainly depends on the refactoring operations that we are interested in supporting. Table 3 reports the algorithms reported in literature, classified by the supported refactoring operation. The algorithms fall in five categories: (i) clustering-based, (ii) graph-based, (iii) search-based, (iv) slicing-based, and (v) heuristic-based.

Clustering algorithms are particularly well suited to support refactoring operations dealing with splitting of complex decomposition units (e.g., Blob classes, promiscuous packages) into simpler ones. In fact, these algorithms have been mainly used to support Extract Class refactoring. The main problem to solve with these algorithms is related to the definition of the number of clusters to form. As example, partitioning algorithms like k -means [31] explicitly require as input the number of clusters to be generated, i.e., the number of classes that should be extracted from the Blob in case of Extract Class refactoring. This means that a developer performing Extract Class refactoring should know a priori the number of different responsibilities implemented in the Blob class, which is not a realistic assumption. Heuristics approximating the optimal number of clusters could be applied (e.g., the silhouette heuristic [60]). However, the performance of these heuristics in the refactoring field is currently unknown. Another possibility are hierarchical agglomerative algorithms, successfully applied by Fokaefs et al. [25]. Such algorithms start by placing each entity to be clustered in a single cluster. Then, at each iteration, merge the two most similar clusters, terminating when all entities are contained in a single cluster. The output of hierarchical agglomerative algorithms is represented with a dendrogram, a tree diagram where the leafs of the tree represent the entities to cluster while the remaining nodes represent possible clusters the entities belong to, up to the root representing a cluster containing all the entities. The distance between merged clusters increases with the level of the merger (starting from the leaves towards the root). This means that nodes (i.e., clusters) at a higher level group together entities having higher distance (lower similarity) between them. In fact, the top node, i.e., the root of the tree, groups all entities in a single cluster, while the bottom nodes (i.e., the leaves) place each entity in a distinct cluster. Finding the right level where to cut the

Table 3: Algorithms exploited in the literature to identify refactoring recommendations

Refactoring Operation	Algorithm Applied	References
Extract Class	Clustering-based	[25, 63]
	Graph-based	[10, 13]
Move Method	Heuristic-based	[5, 52, 66]
	Search-based	[62]
Extract Method	Slicing-based	[1, 67]
Extract Package	Graph-based	[9]
Move Class	Search-based	[3]
	Heuristic-based	[12]
Combination of Multiple Operations	Search-based	[51]

dendrogram (i.e., determine the clusters) is a difficult problem that has to be solved by applying heuristics (e.g., the one defined by Fokaefs et al. [25]).

Other algorithms used to support refactoring aimed at simplifying complex decomposition units (i.e., Extract Class, and Extract Package) are those based on graph theory. First, a weighted-graph representation of the complex object to decompose is built. Each node in the graph represents one of the entities composing the complex object. For example, these components could be the classes of a promiscuous package in need of Extract Package refactoring. As for the weight on the edge connecting two nodes, it can be used to represent the relationships between them (i.e., how similar they are). Having a graph-based representation, it is possible to apply graph partitioning algorithms in order to split the original graph representing the complex object into subgraphs representing the new simpler objects to extract. Examples are the Max Flow-Min Cut algorithm applied by Bavota et al. [10] that, however, suffers of a strong limitation: it always splits the complex object into two new simpler objects. This means that, for example, each Blob class is always split into two new classes, independently from the different responsibilities implemented in it. Another possibility is to identify disconnected components in the graph, as done by Bavota et al. [13]. However, several different choices can be made by looking at graph theory-based algorithms [64]. Generating a unique refactoring solution using graph-based algorithms is generally easier than adopting clustering algorithms. While the solution generated by using graph-based algorithms is unique, they do not provide to the developer alternative solutions like, for example, the hierarchical agglomerative algorithms.

When the refactoring operation to support is Extract Method, slicing-based algorithms represent the most obvious choice. In fact, slicing algorithms allow to decompose a large method (generally the kind of methods targeted for this refactoring) into different smaller methods, each one represented by a slice. Thus, a slicing

algorithm, generally in its static variant, represents the natural solution for this kind of refactoring.

Widely used to support Move Method refactoring are heuristic-based algorithms [5, 24, 52, 66]. When the problem faced by the refactoring operation is to move pieces of code to a more appropriate place, a simple analysis of the dependencies between code components can be enough. For example, to recommend that a method m , implemented in a class C_m , should be moved in a new class C_n , it is enough to:

1. count the dependencies existing between m and C_m . The dependencies to analyze depend on the sources of information exploited by the approach (see Section 3.1).
2. count the dependencies existing between m and C_n .
3. if the dependencies between m and C_n are more than those existing between m and C_m , then recommend moving m in C_n .

Finally, search based algorithms can be applied to all kinds of refactoring operations. Not by chance, they are the only algorithms applied when the goal is to support a combination of multiple refactoring operations, as done by O’Keeffe and O’Cinneide [51]. In these algorithms, the task of refactoring is formulated as a search problem in the space of alternative designs. The alternative designs are generated applying a set of refactoring operations supported by the approach. The search for the optimal design is guided by a quality evaluation function, usually based on code quality metrics. The strengths of these algorithms is their generality. On the weaknesses side, given the non-deterministic component of these algorithms, they may recommend a different refactoring solution at each run. Non-determinism is not a desirable property for a refactoring recommendation system.

4 Evaluating a Refactoring Recommendation System

The evaluation of a refactoring recommendation system represents another challenge. This is due to the fact that, unlike other fields (e.g., traceability recovery, bug prediction, etc.), in most cases no oracles are available to evaluate the performance of the recommendation systems. In this section we report some evaluation strategies that can be useful in the evaluation of refactoring recommendation systems.

4.1 Evaluation based on Quality Metrics

The easiest way to get a first evaluation of a refactoring recommendation system is to exploit quality metrics. In other words, selected quality metrics are measured in a software system before and after the application of the recommended refactoring operations to verify if the internal software quality of the system is improved with the application of the refactoring operations. For instance, there are several quality

Table 4: Evaluations performed in the literature to experiment a refactoring recommendation system

Reference	Supported Refactoring	Quality Metrics	Historical Information	External Developers	Original Developers
[63]	Extract Class				
[25]	Extract Class	X			X
[10]	Extract Class	X		X	
[13]	Extract Class	X	X	X	
[5]	Move Method			X	
[62]	Move Method	X		X	
[66]	Move Method	X			X
[52]	Move Method				
[1]	Extract Method				
[67]	Extract Method	X		X	
[9]	Extract Package			X	X
[3]	Move Class	X			
[12]	Move Class	X		X	X
[51]	Multiple Operations			X	

metrics available in literature that have been demonstrated to capture different aspects of software maintainability. For example, the previously presented MPC metric (see Section 3.1) directly correlates with the maintenance effort [37], i.e., higher MPC values (higher coupling) indicate higher effort in maintaining a software system. Thus, refactoring recommendations reducing the MPC in a software system are certainly preferable to those increasing the MPC. As another example, the CCBC metric (see Section 3.1) has been used to support change impact analysis. Thus, two classes exhibiting high CCBC are likely to be co-changed during a modification activity performed in a system. Consequently, refactoring recommendations (e.g., Move Class refactoring) able to group in the same software module classes having high CCBC between them could reduce the effort needed by a developer to localize the change. This clearly results in more manageable maintenance activities.

Several empirical studies provided evidence that high levels of coupling and lack of cohesion are associated with lower productivity, greater rework, and more design effort for developers [7, 16–18, 21]. In addition, lower cohesion and/or higher coupling of classes have been shown to correlate with higher defect rates [29, 39, 41]. Thus, refactoring recommendation systems should suggest refactoring operations able to improve a software system from the quality metrics point-of-view (e.g., increase cohesion and reduce coupling).

It is important to highlight that, while metrics-based evaluations allow to easily compare different recommendation systems, they are affected by many threats to validity. First, in all the software metrics evaluations there is a risk that the improve-

ments in terms of the quality metrics achieved by applying the proposed refactoring operations are obtained by construction. In fact, the information used by the recommendation systems to identify refactoring opportunities (e.g., method calls, shared instance variables, semantic similarity) are often the same exploited by the quality metrics. For example, all Extract Class recommendation systems exploit information like method calls and shared instance variables to identify cohesive clusters of methods that can be extracted from the Blob class. Since this information is the same used by cohesion metrics (e.g., Lack of Cohesion of Methods [21], which uses information about shared instance variables), the increase of cohesion achieved after performing a recommended Extract Class refactoring is expected. Thus, even if a software metric evaluation is needed to verify that a refactoring technique does not negatively affect the software quality, this kind of evaluation should not be central in the evaluation of a new technique. Besides improving quality metrics it is necessary to show that recommended refactoring operations are meaningful from a developer's point of view. In fact, improvements in quality metrics are not always enough to justify the need to change the original design from the developers' point of view.

Many approaches [3, 10, 12, 13, 25, 62, 66, 67] have been evaluated using quality metrics (see Table 4). Results of these evaluations generally showed improvements in the values of the chosen metrics. For instance, the Extract Class refactoring approach proposed by Bavota et al. [13] was able to increase the cohesion of the refactored Blob classes of over five times. Other approaches, like those for Move Method refactoring, achieve smaller improvements due to the fact that the applied refactoring is quite circumscribed. For example, the approach by Tsantalis and Chatzigeorgiou [66] was able to improve class cohesion by 3%, on average.

4.2 Evaluation based on Historical Information

Developers are not always available for evaluating a refactoring recommendation systems. A possible solution to evaluate the recommended refactoring from a developer's point-of-view, without directly involving developers is to simulate their presence by using as oracle refactoring operations previously performed by developers⁷. These refactoring operations can be obtained by using available tools that able to identify refactoring operations performed among two subsequent version of a software system. For example, the Ref-Finder tool [57] is able to identify 63 different types of refactoring. The tool is not 100% precise, but the refactoring operations retrieved by it can be manually validated in order to obtain a reliable oracle on which to test the refactoring recommendation system. For example, suppose that we want to evaluate a Move Method refactoring recommendation system. We can identify the move method refactoring operations performed by developers of an open source system *S* between version 1.0 and version 1.1 of *S*. Then, we can apply our refac-

⁷ For more information on simulation techniques to evaluate recommendation systems see Chapter ?? [Chapter reference??](#)

toring recommendation system on the version 1.0 of S and measure (e.g., through precision and recall) to what extent the recommended refactoring operations reflect those performed by the original developers. If the recommendation system is able to propose with a good accuracy the refactoring actually performed by original developers this means that: (i) the recommended refactoring operations are (at least by some approximation) meaningful from a developer's point-of-view and (ii) the recommendation systems can effectively support the developers in the identification of refactoring solutions.

To date, this type of evaluation has been only exploited in the evaluation of ARIES [13], used for Extract Class refactoring. In this work, the authors use Ref-Finder [57] to mine the history of six open source systems looking for extract class refactoring operations performed by the original developers. Note that Ref-Finder retrieves Extract Class refactoring operations as a set of Move Method and Move Field operations from the original class to the new extracted classes. Thus, a manual validation has been performed on the sets of Move Method and Move Field refactoring retrieved by Ref-Finder to identify extract class refactoring operations. Then, ARIES has been applied to the classes refactored by the developers to measure how far is the refactoring recommendation from the refactoring performed by the original developers. To this aim, the MoJo eEffectiveness Measure (MoJoFM) [69] has been used. The MoJoFM is a normalized variant of the MoJo distance and it is computed as follows:

$$MoJoFM(A, B) = 1 - \frac{mno(A, B)}{\max(mno(\forall A, B))}$$

where $mno(A, B)$ is the minimum number of *Move* or *Join* operations to perform in order to transform the partition A into B , and $\max(mno(\forall A, B))$ is the maximum possible distance of any partition A from the gold standard partition B . Thus, $MoJoFM$ returns 0 if a clustering algorithm produces the farthest partition away from the gold standard; it returns 1 if a clustering algorithm produces exactly the gold standard.

ARIES achieved a 0.91 MoJoFM value, on average, for the 11 classes on which it has been applied [13].

4.3 Evaluation with Developers

The best scenario for evaluating a refactoring recommendation system is when developers are available to rate the meaningfulness of the recommended refactoring operations (e.g., through a Likert scale [53]).

We distinguish two kinds of studies that could be performed. The first is performed with original developers, i.e., the developers evaluate the refactoring operations on systems they developed in the past. The second is performed with external developers, i.e., the developers evaluate the refactoring operations on systems they do not know. The evaluations performed with original developers are preferred since external developers do not have a deep knowledge of the design of the subject system under analysis and thus may not be aware of some of the design choices that

could appear as suboptimal, but that are the results of a rational choice. However, studies with external developers can complement studies performed with original developers. In fact, even if the original developers have deep knowledge of the system's design, they could be the authors of some bad design choices and consequently could not recognize good recommended refactorings as meaningful. This threat can be mitigated by conducting a study with the external developers.

Several authors have performed evaluations with original developers [25, 66], while external developers have also been involved in several empirical evaluations [5, 10, 51, 62, 67]. Some works have involved both types of developers [9, 12] (see Table 4). Generally, the results achieved in such evaluations showed that most of the refactoring suggestions generated by the experimented tools are appreciated by developers that recognize their meaningfulness. For instance, the approach proposed by Bavota et al. [12] to support Move Class refactoring has been evaluated with 48 external developers and 10 original developers. More than 70% of the suggested Move Class refactoring operations have been judged as meaningful by the subjects.

5 Conclusions and Open Issues

Refactoring is an important activity employed to improve the quality of a software system during its evolution. Different refactoring operations might improve different quality aspects of a system. In particular, there are more than 90 different refactoring operations defined in the Fowler's catalog. Most of the refactoring operations are simple source code transformations aimed at increasing source code comprehension. Others are really complex and their application is error-prone and time consuming. This is the reason why in the last decade a lot of effort has been devoted to the definition of approaches able to recommend non-trivial refactoring operations (e.g., Extract Class refactoring).

In this chapter we have described and analyzed the state of the art, while deriving guidelines for building as well as evaluating a refactoring recommendation system. The analysis of the literature has also allowed to identify some open issues.

Preserving the System Behavior. A first issue is related to the intrinsic definition of refactoring. In theory, a refactoring should not change the behavior of a software system, but only help in improving some of its non-functional attributes. In practice, a refactoring might be risky as any other change occurring in a system, causing possible bug introductions. A recent study investigated the extent to which refactoring activities induce bug fixes in software systems [8]. The authors analyzed a total of 12,922 refactoring operations of 52 different types and found that 15% of the analyzed refactorings likely induced faults in the system. Moreover, the authors found that some specific kinds of refactorings are very likely to induce bug fixes, such as Pull Up Method and Extract Subclass, where the percentage of fixes likely induced by such refactorings is around 40%.

This result could be partially explained by the survey performed by Kim et al. [34] with 328 Microsoft engineers (of which 83% developers) to understand (i) the participants own refactoring definition, (ii) when and how they refactor code, (iii) if refactoring tools are used by developers, and (iv) developers perception toward the benefits, risks, and challenges of refactoring. Three of their findings are related to the problem of preserving system behavior [34]:

- while developers recognize refactoring as a way to improve the quality of a software, in almost 50% of the cases *they do not define refactoring as a behavior-preserving operation*;
- 51% of developers *manually perform refactoring*;
- the *main risk* the developers fear when performing refactoring operations is *bug introduction* (77%).

Some of these findings are also partially confirmed in the study performed by Murphy-Hill et al. [49]. They analyze eight different datasets trying to understand how developers perform refactoring. Examples of the exploited datasets are usage data from 41 developers using the Eclipse environment, data from the Eclipse Usage Collector aggregating activities of 13,000 developers for almost one year, and information extracted from versioning systems. The paper discusses several interesting findings: (i) developers often interleave refactoring with other programming activities (thus, potentially changing the system behavior), and (ii) most of the refactoring operations (close to 90%) are performed manually by developers without the help of any tool.

While the fact that developers mostly perform refactoring manually could explain the bugs introduced through refactoring as observed by Bavota et al. [8]. Daniel et al. [22] showed that even automated refactoring performed by Integrated Development Environments could be fault-prone. They propose a possible solution to ensure that refactoring engines (i.e., the tools that apply the refactoring operations) correctly transform the program when applying refactoring operations. In particular, Daniel et al. [22] proposed an approach to automate the testing of refactoring engines. The technique generates abstract syntax trees of the refactored Java programs (to date the only supported programming language) to verify that the behavior of the system remains unchanged after the application of the refactoring operations. Their approach was able to identify 45 previously unreported bugs in Eclipse and NetBeans, two of the most popular refactoring engines for Java [22]. Thus, the approach by Daniel et al. [22] certainly represent a good way to verify that a refactoring recommendation tool correctly applies refactoring operations.

Solutions to the problem of behavior preservation during refactoring have been also proposed by Overbey and Johnson [54], Schäfer et al. [61], and Mens et al. [46]. However, none of these works have presented a fully generalizable approach that can be applied for every possible refactoring.

Another possibility to ensure that the performed refactoring does not alter the system external behavior is to define strong pre-conditions that must be verified in order to apply it. An example of this approach is in the technique for Move Method refactoring presented by Tsantalis and Chatzigeorgiou [66]. The Move Method

refactoring operation is performed only if the involved code components satisfy all preconditions. However, this “strong pre-conditions philosophy” has also been criticized in literature [59] since it could avoid the application of desirable refactoring operations just because there is a small price to pay in terms of small adjustments in the source code. The result of this observation has been the *program metamorphosis* approach proposed by Reichenbach et al. [59] that relaxes the behavior-preservation checks by considering them later in the refactoring process: when a refactoring alters the external behavior of the system, the approach allows the user to correct or accept the behavioral changes.

All in all, the problem of ensuring the preservation of the system behavior during refactoring has still to be solved.

Usability of the Refactoring Recommendation Systems. A second open issue is related to the usability of the refactoring recommendation system. In general, recommendations are based only on quality metrics. However, quality metrics do not tell the whole story about the code and may not be able to capture the perception of the developer about refactoring. This could generate refactoring operations that are not usable because the developer is not able to understand why the recommendation system is proposing a specific refactoring operations. In such a situation the developer might lose faith in the recommendation system. A way to overcome such a situation has been recently proposed by Bavota et al. [12]. In particular, the authors analyze underlying natural language topics in classes not only to identify refactoring operations but also to identify their responsibilities and provide some rationale behind the proposed refactoring recommendation. An empirical study conducted with developers has indicated the usefulness of such an explanation in understanding (and, consequently, in accepting or rejecting) a recommended refactoring operation [12]. However, the approach exploited by Bavota et al. [12] represents a first attempt, while more sophisticated techniques could be used to automatically derive the rationale behind a recommended refactoring operation (e.g., by exploiting natural language techniques).

Provide a Complete Support to the Developer Performing Refactoring. As of today, there are no approaches that provide simultaneous and integrated support to multiple types of refactoring operations. It would be worthwhile to have tools able to suggest a complete set of refactoring operations of different types able to improve the design of a software system from different points of view (e.g., removal of different types of antipatterns, improved adherence to object-oriented design practices, etc.). This would also allow to integrate in the refactoring tool a wider subset of existing refactoring operations. As example, recommending Rename Method refactoring solutions could be worthwhile if combined with Extract Method recommendations. In fact, automatically assigning a name reflecting the responsibility implemented in the extracted method can save developers’ time. Note that in the literature the goal of covering more refactoring operations is accomplished in part in the work by O’Keeffe and O’Cinneide [51] where, however, only refactoring operations working on class hierarchies are combined together.

Also, most of the approaches in the literature have not been implemented in IDEs and this does not encourages developers in using them, as revealed in the existing studies [34, 49]. While the integration of refactoring approaches in IDEs is far from trivial for the reasons discussed above (i.e., behavior preservation, usability of the tools), more effort should be spent in this direction.

References

1. Abadi, A., Ettinger, R., Feldman, Y.A.: Fine slicing for advanced method extraction. In: 3rd Workshop on Refactoring Tools (2009)
2. Abadi, A., Ettinger, R., Feldman, Y.A.: Fine slicing: theory and applications for computation extraction. In: Proceedings of the 15th international conference on Fundamental Approaches to Software Engineering, FASE'12, pp. 471–485. Springer-Verlag, Berlin, Heidelberg (2012)
3. Abdeen, H., Ducasse, S., Sahraoui, H.A., Alloui, I.: Automatic package coupling and cycle minimization. In: Proceedings of the 16th Working Conference on Reverse Engineering, pp. 103–112. IEEE CS Press, Lille, France (2009)
4. Abebe, S., Tonella, P.: Automated identifier completion and replacement. In: Software Maintenance and Reengineering (CSMR), 2013 17th European Conference on, pp. 263–272 (2013)
5. Atkinson, D.C., King, T.: Lightweight detection of program refactorings. In: Proceedings of 12th Asia-Pacific Software Engineering Conference, pp. 663–670. IEEE CS Press, Taipei, Taiwan (2005)
6. Baeza-Yates, R., Ribeiro-Neto, B.: Modern Information Retrieval. Addison-Wesley (1999)
7. Basili, V.R., Briand, L., Melo, W.L.: A validation of object-oriented design metrics as quality indicators. *IEEE Transactions on Software Engineering* **22**(10), 751–761 (1995)
8. Bavota, G., Carluccio, B.D., Lucia, A.D., Penta, M.D., Oliveto, R., Strollo, O.: When does a refactoring induce bugs? an empirical study. In: Proceedings of the 12th International Working Conference on Source Code Analysis and Manipulation (SCAM 2012), pp. 104–113 (2012)
9. Bavota, G., De Lucia, A., Marcus, A., Oliveto, R.: Using structural and semantic measures to improve software modularization. *Empirical Software Engineering* p. To appear (2013)
10. Bavota, G., De Lucia, A., Oliveto, R.: Identifying extract class refactoring opportunities using structural and semantic cohesion measures. *Journal of Systems and Software* **84**, 397–414 (2011)
11. Bavota, G., Dit, B., Oliveto, R., Penta, M.D., Poshyvanyk, D., Lucia, A.D.: An empirical study on the developers perception of software coupling. In: Proceedings of the 35th International Conference on Software Engineering, ICSE 2013, p. To appear. (2013)
12. Bavota, G., Gethers, M., Oliveto, R., Poshyvanyk, D., De Lucia, A.: Improving software modularization via automated analysis of latent topics and dependencies. *Transactions on Software Engineering and Methodologies* p. To appear (2013)
13. Bavota, G., Lucia, A.D., Marcus, A., Oliveto, R.: Automating extract class refactoring: an improved method and its evaluation. In: *Empirical Software Engineering*, p. To appear (2013)
14. Bavota, G., Lucia, A.D., Marcus, A., Oliveto, R., Palomba, F.: Supporting extract class refactoring in eclipse: The aries project. In: ICSE, pp. 1419–1422 (2012)
15. Beck, K., Andres, C.: *Extreme Programming Explained: Embrace Change*. Addison-Wesley Professional (2004)
16. Binkley, A.B., Schach, S.R.: Validation of the coupling dependency metric as a predictor of run-time failures and maintenance measures. In: Proceedings of the 20th International Conference on Software Engineering, pp. 452–455. Kyoto, Japan (1998)
17. Briand, L.C., Wuest, J., Lounis, H.: Using coupling measurement for impact analysis in object-oriented systems. In: Proceedings of the 15th IEEE International Conference on Software Maintenance, pp. 475–482. IEEE Press, Oxford, UK (1999)

18. Briand, L.C., Wüst, J., Ikonovskii, S.V., Lounis, H.: Investigating quality factors in object-oriented designs: an industrial case study. In: *Proceedings of the 21st International Conference on Software Engineering*, pp. 345–354. ACM Press, Los Angeles, California, United States (1999)
19. Brown, W.J., Malveau, R.C., Brown, W.H., McCormick III, H.W., Mowbray, T.J.: *Anti Patterns: Refactoring Software, Architectures, and Projects in Crisis*. 1st edn. John Wiley and Sons (1998)
20. Chang, J., Blei, D.M.: Hierarchical relational models for document networks. *Annals of Applied Statistics* (2010)
21. Chidamber, S.R., Kemerer, C.F.: A metrics suite for object oriented design. *Transactions on Software Engineering* **20**(6), 476–493 (1994)
22. Daniel, B., Dig, D., Garcia, K., Marinov, D.: Automated testing of refactoring engines. In: *Proceedings of the 6th joint meeting of the European Software Engineering Conference and the ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2007, Dubrovnik, Croatia, September 3–7, 2007, pp. 185–194. ACM (2007)
23. Deerwester, S., Dumais, S.T., Furnas, G.W., Landauer, T.K., Harshman, R.: Indexing by latent semantic analysis. *Journal of the American Society for Information Science* **41**(6), 391–407 (1990)
24. Du Bois, B., Demeyer, S., Verelst, J.: Refactoring - improving coupling and cohesion of existing code. In: *Proceedings of 11th Working Conference on Reverse Engineering*, pp. 144–151. IEEE CS Press, Delft, the Netherlands (2004)
25. Fokaefs, M., Tsantalis, N., Chatzigeorgiou, A., Sander, J.: Decomposing object-oriented class modules using an agglomerative clustering technique. In: *Proceedings of the 25th International Conference on Software Maintenance*, pp. 93–101. Edmonton, Canada (2009)
26. Fokaefs, M., Tsantalis, N., Stroulia, E., Chatzigeorgiou, A.: Identification and application of extract class refactorings in object-oriented systems. *Journal of Systems and Software* **85**(10), 2241–2260 (2012)
27. Fowler, M.: *Refactoring: improving the design of existing code*. Addison-Wesley (1999)
28. Gui, G., Scott, P.D.: Coupling and cohesion measures for evaluation of component reusability. In: *Proceedings of the 5th International Workshop on Mining Software Repositories*, pp. 18–21. ACM Press, Shanghai, China (2006)
29. Gyimóthy, T., Ferenc, R., Siket, I.: Empirical validation of object-oriented metrics on open source software for fault prediction. *IEEE Transactions on Software Engineering* **31**(10), 897–910 (2005)
30. Harman, M., Hierons, R.M., Proctor, M.: A new representation and crossover operator for search-based optimization of software modularization. In: *Proceedings of the Genetic and Evolutionary Computation Conference*. Morgan Kaufmann Publishers Inc., New York, USA (2002)
31. Hartigan, J.A.: *Clustering Algorithms*. Wiley (1975)
32. Kerievsky, J.: *Refactoring to Patterns*. Pearson Higher Education (2004)
33. Khomh, F., Vaucher, S., Guéhéneuc, Y.G., Sahraoui, H.: Bdtex: A gqm-based bayesian approach for the detection of antipatterns. *Journal of Systems and Software* pp. 559–572 (2011)
34. Kim, M., Zimmermann, T., Nagappan, N.: A field study of refactoring challenges and benefits. In: *Proceedings of the 20th International Symposium on Foundations of Software Engineering* (2012)
35. Lanza, M., Marinescu, R.: *Object-Oriented Metrics in Practice: Using Software Metrics to Characterize, Evaluate, and Improve the Design of Object-Oriented Systems*. Springer (2006)
36. Lawrie, D., Morrell, C., Feild, H., Binkley, D.: Effective identifier names for comprehension and memory. *ISSE* **3**(4), 303–318 (2007)
37. Lee, Y., Liang, B., Wu, S., Wang, F.: Measuring the coupling and cohesion of an object-oriented program based on information flow. In: *Proceedings of International Conference on Software Quality*, pp. 81–90. Maribor, Slovenia (1995)
38. Li, W., Henry, S.: Maintenance metrics for the object oriented paradigm. In: *Proc. of MET-RICS*, pp. 52–60 (1993)

39. Liu, Y., Poshyvanyk, D., Ferenc, R., Gyimóthy, T., Chrisochoides, N.: Modeling class cohesion as mixtures of latent topics. In: *Proceedings of 25th IEEE International Conference on Software Maintenance*, pp. 233–242. IEEE CS Press, Edmonton, Canada (2009)
40. Mancoridis, S., Mitchell, B.S., Rorres, C., Chen, Y.F., Gansner, E.R.: Using automatic clustering to produce high-level system organizations of source code. In: *Proceedings of 6th International Workshop on Program Comprehension*. IEEE CS Press, Ischia, Italy (1998)
41. Marcus, A., Poshyvanyk, D., Ferenc, R.: Using the conceptual cohesion of classes for fault prediction in object-oriented systems. *IEEE Transaction on Software Engineering* **34**(2), 287–300 (2008)
42. Marinescu, R.: Detection strategies: Metrics-based rules for detecting design flaws. In: *Proceedings of the 20th IEEE International Conference on Software Maintenance*, pp. 350–359. IEEE Computer Society, Washington, DC, USA (2004)
43. Martin, R.C.: *Clean Code: A handbook of agile software craftsmanship*. Prentice Hall (2009)
44. Maruyama, K., Shima, K.: Automatic method refactoring using weighted dependence graphs. In: *Proceedings of 21st International Conference on Software Engineering*, pp. 236–245. ACM Press, Los Alamitos, California, USA (1999)
45. Mens, T., Tourwe, T.: A survey of software refactoring. *IEEE Transactions on Software Engineering* **30**(2), 126–139 (2004)
46. Mens, T., Van Eetvelde, N., Demeyer, S., Janssens, D.: Formalizing refactorings with graph transformations: Research articles. *J. Softw. Maint. Evol.* **17**(4), 247–276 (2005)
47. Mitchell, B.S., Mancoridis, S.: On the automatic modularization of software systems using the bunch tool. *IEEE Trans. Software Eng.* **32**(3), 193–208 (2006)
48. Moha, N., Gueheneuc, Y.G., Duchien, L., Le Meur, A.F.: Decor: A method for the specification and detection of code and design smells. *IEEE Transactions on Software Engineering* **36**(1), 20–36 (2010)
49. Murphy-Hill, E., Parnin, C., Black, A.P.: How we refactor, and how we know it. *Transactions on Software Engineering* **38**(1), 5–18 (2011)
50. Nierstrasz, O., Ducasse, S., Demeyer, S.: *Object-Oriented Reengineering Patterns*. Morgan Kaufmann Publishers Inc. (2003)
51. O’Keeffe, M., O’Cinneide, M.: Search-based software maintenance. In: *Proceedings of 10th European Conference on Software Maintenance and Reengineering*, pp. 249–260. IEEE CS Press, Bari, Italy (2006)
52. Oliveto, R., Gethers, M., Bavota, G., Poshyvanyk, D., De Lucia, A.: Identifying method friendships to remove the feature envy bad smell. In: *Proceedings of the 33rd IEEE/ACM International Conference on Software Engineering*. ACM Press, Hawaii, USA (2011)
53. Oppenheim, A.N.: *Questionnaire Design, Interviewing and Attitude Measurement*. Pinter Publishers (1992)
54. Overbey, J., Johnson, R.: Differential precondition checking: A lightweight, reusable analysis for refactoring tools. In: *Automated Software Engineering (ASE), 2011 26th IEEE/ACM International Conference on*, pp. 303–312 (2011)
55. Poshyvanyk, D., Marcus, A., Ferenc, R., Gyimóthy, T.: Using information retrieval based coupling measures for impact analysis. *Empirical Software Engineering* **14**(1), 5–32 (2009)
56. Praditwong, K., Harman, M., Yao, X.: Software module clustering as a multi-objective search problem. *IEEE Transactions on Software Engineering* **37**(2), 264–282 (2011)
57. Prete, K., Rachatasumrit, N., Sudan, N., Kim, M.: Template-based reconstruction of complex refactorings. In: *26th IEEE International Conference on Software Maintenance (ICSM 2010)*, September 12–18, 2010, Timisoara, Romania, pp. 1–10. IEEE Computer Society (2010)
58. Rao, A.A., Reddy, K.: Detecting bad smells in object oriented design using design change propagation probability matrix. In: *Proceedings of the International MultiConference of Engineers and Computer Scientists. Lecture Notes in Engineering and Computer Science* (2007)
59. Reichenbach, C., Coughlin, D., Diwan, A.: Program metamorphosis. In: *Proceedings of the 23rd European Conference on ECOOP 2009 — Object-Oriented Programming*, Genoa, pp. 394–418. Springer-Verlag (2009)

60. Rousseeuw, P.J.: Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics* **20**(0), 53 – 65 (1987)
61. Schäfer, M., Verbaere, M., Ekman, T., Moor, O.: Stepping stones over the refactoring rubicon. In: *Proceedings of the 23rd European Conference on ECOOP 2009 — Object-Oriented Programming*, Genoa, pp. 369–393 (2009)
62. Seng, O., Stammel, J., Burkhart, D.: Search-based determination of refactorings for improving the class structure of object-oriented systems. In: *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 1909–1916. Seattle, Washington, USA (2006)
63. Simon, F., Steinbr, F., Lewerentz, C.: Metrics based refactoring. In: *Proceedings of 5th European Conference on Software Maintenance and Reengineering*, pp. 30–38. IEEE CS Press, Lisbon, Portugal (2001)
64. Thulasiraman, K., Swamy, M.N.S.: *Graphs: theory and algorithms*. John Wiley & Sons, Inc. (1992)
65. Travassos, G., Shull, F., Fredericks, M., Basili, V.: Detecting defects in object-oriented designs: using reading techniques to increase software quality. In: *Proceedings of the 14th Conference on Object-Oriented Programming, Systems, Languages, and Applications*, pp. 47–56. ACM Press (1999)
66. Tsantalis, N., Chatzigeorgiou, A.: Identification of move method refactoring opportunities. *IEEE Transactions on Software Engineering* **35**, 347–367 (2009)
67. Tsantalis, N., Chatzigeorgiou, A.: Identification of extract method refactoring opportunities for the decomposition of methods. *Journal of Systems and Software* **84**(10), 1757–1782 (2011)
68. Weiser, M.: Program slicing. *IEEE Transactions on Software Engineering* **10**(4), 352–357 (1984)
69. Wen, Z., Tzerpos, V.: An effectiveness measure for software clustering algorithms. In: *Proceedings of the 12th IEEE International Workshop on Program Comprehension*, pp. 194–203 (2004)
70. Wu, J., Hassan, A.E., Holt, R.C.: Comparison of clustering algorithms in the context of software evolution. In: *Proceedings of 21st IEEE International Conference on Software Maintenance*, pp. 525–535. IEEE CS Press, Budapest, Hungary (2005)